

EduBatch

Project Documentation Package

A complete technical specification for a small-scale
Education Batch Management Platform

Document Type	Technical Specification & Developer Guide
Version	1.0.0
Status	Ready for Implementation
Stack	Node.js · Express · MongoDB · React · Razorpay
Audience	Developers, Project Managers, Stakeholders

Table of Contents

1. Project Idea
2. Requirements
3. Features
4. User Roles
5. System Architecture
6. Database Design
7. API Documentation
8. Project Folder Structure
9. Development Steps
10. Security
11. Payment Flow
12. Deployment
13. GitHub Workflow
14. Roadmap
15. Testing Checklist

1. Project Idea

EduBatch is a lightweight, web-based Education Batch Management Platform designed for coaching institutes, tuition centers, and small educational organizations. It solves the common pain points of managing student batches, tracking enrollments, collecting fees, recording attendance, and communicating with students and parents — all from a single dashboard.

Unlike heavy ERP systems, EduBatch focuses on a minimal, fast, and developer-friendly core that can be built and deployed by a small team (or even a solo developer) within a few weeks. The platform supports multi-role access, online payments, and a clean RESTful API that can later power mobile apps.

Core Value Proposition

- Centralized batch & student management instead of spreadsheets
- Online fee collection with automated receipt generation
- Role-based dashboards for Admin, Teacher, and Student
- Attendance tracking and basic progress visibility
- Simple, modern UI that works on desktop and mobile browsers

Target users: coaching centers (JEE/NEET/Board), language institutes, coding bootcamps, and small private schools that need a focused batch-centric tool rather than a full school ERP.

2. Requirements

2.1 Functional Requirements

- User authentication with JWT (login / register / logout / password reset)
- Admin can create, edit, archive batches (name, subject, schedule, capacity, fee)
- Admin / Teacher can enroll or remove students from batches
- Students can view their enrolled batches, fee status, and attendance
- Online payment for batch fees (Razorpay integration)
- Attendance marking by Teacher or Admin (date-wise)
- Basic announcements / notices per batch
- Dashboard analytics: total students, revenue, active batches, pending fees
- Profile management for all roles

2.2 Non-Functional Requirements

- Responsive UI (mobile-first where practical)
- API response time < 300 ms for common endpoints under normal load
- Secure password storage (bcrypt) and HTTPS in production
- Horizontal scalability of the API layer (stateless JWT)
- Clear error messages and consistent HTTP status codes
- Codebase maintainable by a junior-to-mid developer

2.3 Tech Stack

Layer	Technology	Rationale
Frontend	React 18 + Vite + Tailwind CSS	Fast DX, modern component model
Backend	Node.js + Express	Lightweight, large ecosystem
Database	MongoDB + Mongoose	Flexible schema for educational data
Auth	JWT + bcrypt	Stateless, industry standard
Payments	Razorpay	Popular in India, good docs & test mode
Hosting	Vercel (FE) + Render / Railway (BE)	Simple free/paid tiers
Email	Nodemailer / Resend	Transactional emails (receipts, resets)

3. Features

Authentication & Authorization

Register, login, JWT refresh, role-based route guards, password reset via email token.

Batch Management

CRUD for batches — name, subject, start/end date, schedule (days & time), max capacity, fee amount, status (upcoming / active / archived).

Student Enrollment

Enroll student into one or more batches. Prevent over-capacity. Track enrollment date and payment status.

Fee & Payment

Generate fee invoice per enrollment. Pay online via Razorpay. Auto-update payment status. Download receipt PDF.

Attendance

Teacher marks present / absent / late for a batch on a given date. Student can view own attendance percentage.

Notices / Announcements

Admin or Teacher posts a notice linked to a batch. Students see notices on their dashboard.

Dashboards

Role-specific overview cards: counts, revenue charts (simple), upcoming classes, pending payments.

Profile

Update name, phone, avatar. Change password. View enrolled batches (student) or assigned batches (teacher).

4. User Roles

Role	Permissions
Admin	Full system access: manage users, create/edit/delete batches, enroll students, view all payments & attendance, post notices, view analytics.
Teacher	View assigned batches, mark attendance, post notices for own batches, view student list of assigned batches. Cannot manage fees or other teachers.
Student	View enrolled batches, pay fees, view own attendance & notices, update own profile. Read-only for everything else.

Roles are stored as a string enum on the User document (admin | teacher | student). Middleware checks `req.user.role` before allowing access to protected routes.

5. System Architecture

EduBatch follows a classic three-tier architecture with a clear separation of concerns.

High-Level Components

- **Client (React SPA)** — served via CDN / Vercel. Communicates only via HTTPS REST calls.
- **API Server (Express)** — stateless, JWT-authenticated. Handles business logic, validation, and orchestration.
- **Database (MongoDB)** — single cluster; collections for users, batches, enrollments, payments, attendance, notices.
- **Payment Gateway (Razorpay)** — server creates order; client completes checkout; webhook confirms success.
- **Email Service** — transactional emails for password reset, payment receipts, enrollment confirmation.

Request Flow (typical authenticated action)

1. Client sends request with `Authorization: Bearer <token>`
2. Express middleware verifies JWT and attaches `req.user`
3. Role guard checks permission for the route
4. Controller validates input (Joi / Zod), calls service layer
5. Service interacts with Mongoose models / external APIs
6. Response returned as JSON with consistent shape: `{ success, data, message }`

Architecture Diagram (textual):



6. Database Design

MongoDB is chosen for flexibility. All collections use Mongoose schemas with timestamps. IDs are ObjectIds. Soft deletes are preferred where history matters (e.g., enrollment).

6.1 users

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Unique, required, lowercase
password	String	Hashed with bcrypt
role	String	enum: admin teacher student
phone	String	Optional
avatar	String	URL (Cloudinary / local)
isActive	Boolean	Default true
createdAt / updatedAt	Date	Auto via timestamps

6.2 batches

Field	Type	Notes
_id	ObjectId	Primary key
name	String	e.g. "JEE 2027 Morning"
subject	String	Physics / Math / Full Course
description	String	Optional long text
startDate / endDate	Date	Batch duration
schedule	Object	{ days: [], startTime, endTime }
capacity	Number	Max students
fee	Number	Amount in INR
teacher	ObjectId	ref → users
status	String	upcoming active archived
createdBy	ObjectId	ref → users (admin)

6.3 enrollments

Field	Type	Notes
_id	ObjectId	Primary key
student	ObjectId	ref → users
batch	ObjectId	ref → batches
enrolledAt	Date	Default now
paymentStatus	String	pending paid waived
payment	ObjectId	ref → payments (nullable)

Field	Type	Notes
isActive	Boolean	Soft leave / drop

6.4 payments

Field	Type	Notes
_id	ObjectId	Primary key
enrollment	ObjectId	ref → enrollments
student	ObjectId	ref → users
amount	Number	In paise for Razorpay
currency	String	INR
razorpayOrderId	String	From create-order
razorpayPaymentId	String	After success
razorpaySignature	String	For verification
status	String	created paid failed
paidAt	Date	Nullable

6.5 attendance

Field	Type	Notes
_id	ObjectId	Primary key
batch	ObjectId	ref → batches
date	Date	Normalized to day
records	Array	[{ student, status: present absent late }]
markedBy	ObjectId	ref → users

6.6 notices

Field	Type	Notes
_id	ObjectId	Primary key
batch	ObjectId	ref → batches (or null for global)
title	String	Required
body	String	Markdown or plain text
createdBy	ObjectId	ref → users
pinned	Boolean	Default false

7. API Documentation

Base URL (dev): `http://localhost:5000/api/v1`

All protected routes require header: `Authorization: Bearer <accessToken>`

7.1 Auth

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register new user (role forced to student unless admin creates)
POST	/auth/login	Public	Returns access + refresh tokens
POST	/auth/refresh	Public	Exchange refresh token for new access token
POST	/auth/forgot-password	Public	Send reset email
POST	/auth/reset-password	Public	Reset with token
GET	/auth/me	Yes	Current user profile

7.2 Batches

Method	Endpoint	Auth	Description
GET	/batches	Yes	List batches (filter by status, teacher)
GET	/batches/:id	Yes	Single batch details + enrolled count
POST	/batches	Admin	Create batch
PUT	/batches/:id	Admin	Update batch
PATCH	/batches/:id/status	Admin	Change status
DELETE	/batches/:id	Admin	Soft archive

7.3 Enrollments & Payments

Method	Endpoint	Auth	Description
POST	/enrollments	Admin	Enroll student in batch
GET	/enrollments/my	Student	My enrollments
POST	/payments/create-order	Student	Create Razorpay order
POST	/payments/verify	Student	Verify signature & mark paid
GET	/payments/history	Yes	Payment history (role-filtered)

7.4 Attendance & Notices

Method	Endpoint	Auth	Description
POST	/attendance	Teacher+	Mark attendance for a date
GET	/attendance/batch/:id	Yes	Attendance records for batch
GET	/attendance/my	Student	Own attendance summary
POST	/notices	Teacher+	Create notice
GET	/notices	Yes	List notices (filter by batch)

Standard response shape: `{ "success": true|false, "data": {...}, "message": "..."}`

8. Project Folder Structure

Monorepo-style layout recommended (or separate repos). Backend example:

```
edubatch/
├── client/                                # React + Vite
│   ├── public/
│   └── src/
│       ├── api/                          # axios instances & endpoint functions
│       ├── components/                   # reusable UI
│       ├── pages/                        # route-level pages
│       ├── hooks/
│       ├── context/                      # AuthContext
│       ├── utils/
│       ├── App.jsx
│       ├── package.json
│       └── vite.config.js
│
├── server/                               # Express API
│   ├── src/
│   │   ├── config/                       # db, env, razorpay
│   │   ├── models/                       # Mongoose schemas
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── middleware/                   # auth, role, validate, error
│   │   ├── services/                     # business logic, email, payment
│   │   ├── utils/                         # helpers, AppError
│   │   └── app.js / server.js
│   ├── package.json
│   └── .env.example
│
├── docs/                                # this documentation
└── README.md
```

9. Development Steps

Recommended order for a solo developer or small team (approx. 3–5 weeks full-time):

- 1. Week 1 — Foundation:** Project setup (Vite + Express + MongoDB Atlas). Auth (register/login/JWT). Basic User model & protected route middleware.
- 2. Week 2 — Core Domain:** Batch CRUD. Enrollment logic + capacity checks. Simple Admin & Student dashboards (list views).
- 3. Week 3 — Payments & Attendance:** Razorpay create-order + verify flow. Attendance marking UI + API. Notices.
- 4. Week 4 — Polish:** Teacher role permissions. Profile pages. Email receipts. Error handling & loading states. Responsive tweaks.
- 5. Week 5 — Deploy & Test:** Environment configs. Deploy FE + BE. End-to-end testing. Seed data script. Documentation hand-off.

Tip: Keep a simple Kanban board (GitHub Projects or Trello) with columns: Backlog → In Progress → Review → Done.

10. Security

- **Passwords:** bcrypt with salt rounds ≥ 10 . Never store plain text.
- **JWT:** Short-lived access tokens (15–30 min) + longer-lived refresh tokens stored in httpOnly cookie or secure storage. Rotate on refresh.
- **Input validation:** Validate all body/query/params with Joi or Zod before business logic.
- **Rate limiting:** Apply express-rate-limit on auth endpoints (e.g., 10 requests / 15 min per IP).
- **Helmet:** Use helmet middleware for secure headers.
- **CORS:** Restrict origin to frontend domain in production.
- **Payment verification:** Always verify Razorpay signature server-side before marking payment as paid.
- **Environment secrets:** Never commit .env. Use platform secret managers (Render, Vercel, Railway).
- **Role checks:** Enforce on every sensitive route; never trust client-side role alone.
- **HTTPS:** Enforce in production. Redirect HTTP → HTTPS.

11. Payment Flow

Razorpay Checkout is used. Flow is designed so the server remains the source of truth.

1. Student clicks “Pay Now” on an enrollment with status = pending.
2. Frontend calls `POST /payments/create-order` with `enrollmentId`.
3. Server creates a Razorpay Order (amount in paise), saves a Payment document with status = created, returns `orderId` + key to client.
4. Client opens Razorpay Checkout modal with the `orderId`.
5. On success, Razorpay returns `paymentId` + signature to the client handler.
6. Client calls `POST /payments/verify` with `orderId`, `paymentId`, signature, `enrollmentId`.
7. Server verifies the signature using the webhook secret / key secret. If valid: update Payment → paid, Enrollment → paid, send receipt email.
8. Client shows success UI and refreshes enrollment list.

Optional: configure a Razorpay webhook endpoint for extra reliability (status updates even if client closes browser).

12. Deployment

Recommended free/cheap stack for a small project:

Component	Platform	Notes
Frontend	Vercel	Connect GitHub repo, auto-deploy on main. Env vars for API URL.
Backend	Render or Railway	Web service from GitHub. Add MongoDB connection string & secrets.
Database	MongoDB Atlas	Free M0 cluster is sufficient for early stage.
Email	Resend or Gmail SMTP	Resend free tier is generous for transactional mail.
Domain	Namecheap / Cloudflare	Point A/CNAME records; enable SSL.

Checklist before going live

- All secrets moved to platform environment variables
- CORS origin set to production frontend URL
- Razorpay switched from test keys to live keys
- MongoDB Atlas IP whitelist / VPC configured
- Error logging in place (e.g., simple console + optional Sentry)
- Seed script run or initial admin created manually

13. GitHub Workflow

Keep the process lightweight yet disciplined.

- **main** — production-ready. Protected; only merge via PR.
- **develop** (optional) — integration branch for ongoing work.
- **feature/*** — short-lived branches: feature/auth, feature/payments, fix/attendance-bug.
- Commit messages: conventional style preferred (feat:, fix:, docs:, chore:).
- Pull Request required for main. At least one approval or self-review checklist.
- Simple GitHub Actions (optional): lint + test on PR; deploy on merge to main.
- Use GitHub Issues or Projects for task tracking.
- Never commit .env, node_modules, or build artifacts. Proper .gitignore is mandatory.

Example branch flow: feature/payment-verify → PR → develop → PR → main → auto-deploy

14. Roadmap

Phase 1 — MVP (current scope)

- Auth, Batches, Enrollments, Payments, Attendance, Notices, basic Dashboards

Phase 2 — Usability & Growth

- Parent / Guardian role (view child's attendance & fees)
- Export attendance & fee reports (CSV / PDF)
- Batch calendar view
- In-app notifications (or simple email digests)
- Student progress / marks module (optional exams)

Phase 3 — Scale & Mobile

- Native mobile app (React Native) using the same API
- Multi-institute / multi-tenant support
- Advanced analytics & revenue charts
- WhatsApp / SMS reminders for fees & classes
- Role-based granular permissions beyond the three roles

15. Testing Checklist

Use this checklist before each release. Mark items as you complete them.

Authentication

- Register with valid / invalid / duplicate email
- Login with correct / incorrect credentials
- Access protected route without token → 401
- Access admin route as student → 403
- Password reset flow end-to-end
- Token expiry & refresh behavior

Batches & Enrollment

- Admin creates / edits / archives a batch
- Cannot enroll when capacity is full
- Student sees only own enrollments
- Teacher sees only assigned batches

Payments

- Create order returns valid Razorpay orderId
- Successful payment updates enrollment status to paid
- Tampered signature is rejected
- Receipt email is received (or logged in test mode)

Attendance & Notices

- Teacher can mark attendance for own batch only
- Student can view own attendance percentage
- Notice appears for enrolled students

General

- All API responses follow the standard shape
- No sensitive data (password, tokens) in responses or logs
- UI works on mobile viewport (key pages)
- 404 and 500 pages / messages are user-friendly

— End of EduBatch Documentation Package —

Generated for implementation readiness. Update version number and changelog as the project evolves.